

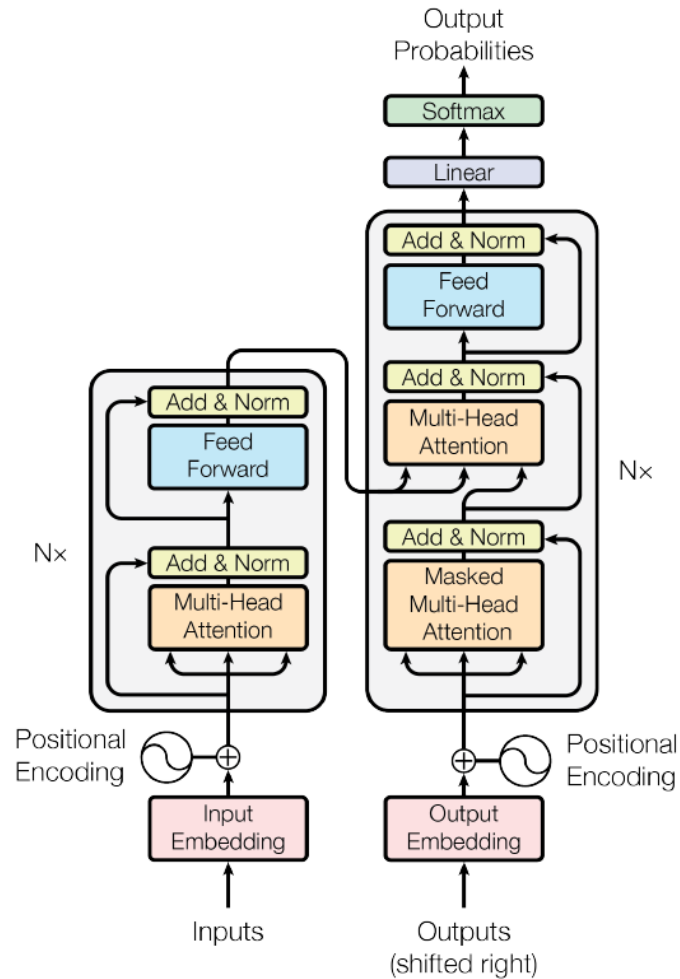


Figure 1: The Transformer - model architecture.

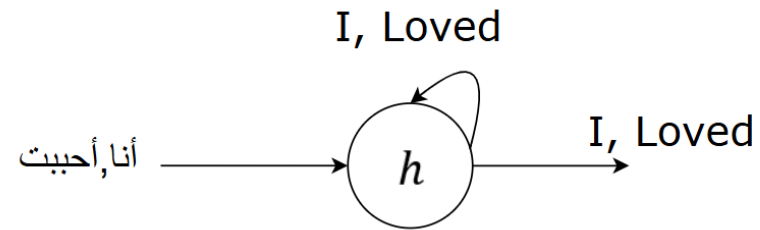
Transformers Architecture

“Attention is all you need”

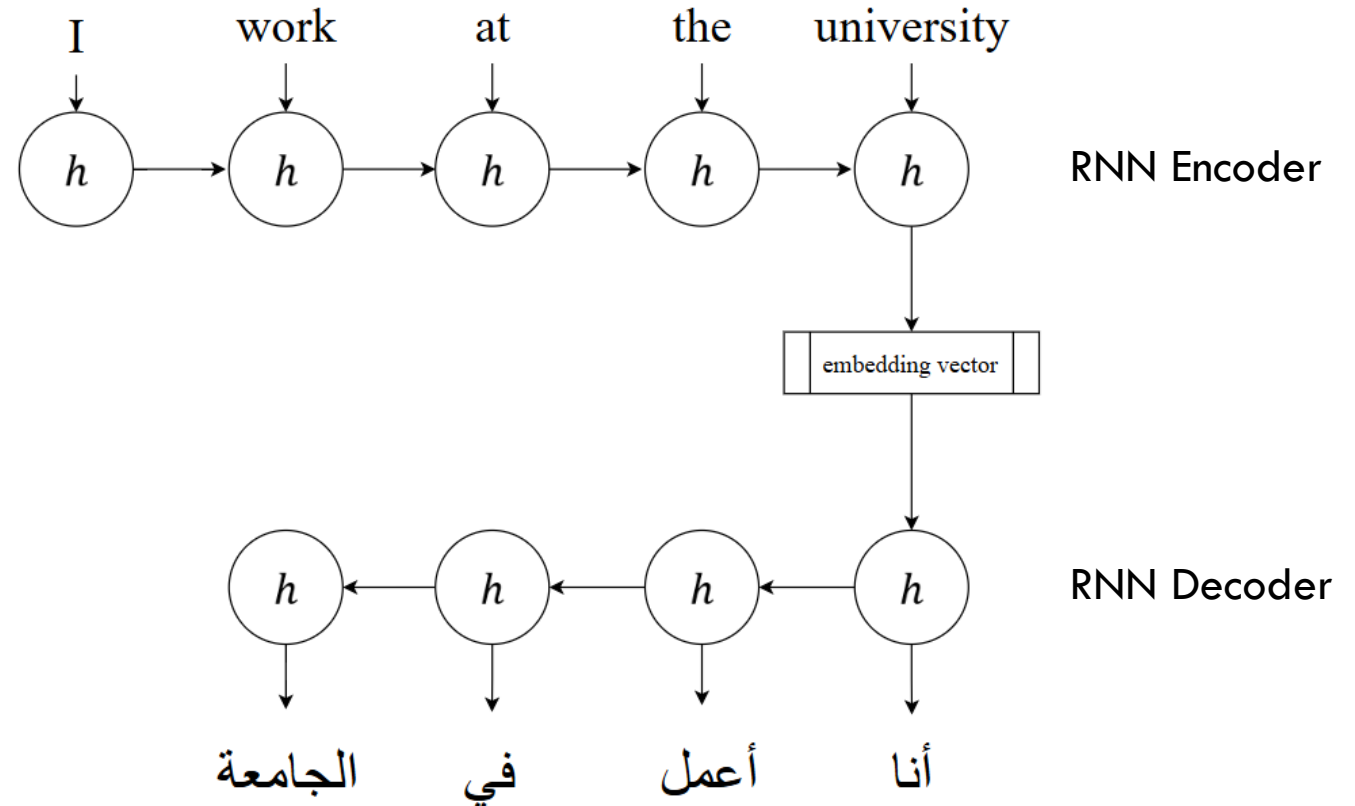
obada mansour

RNN – Example

“I loved this music a lot “



This h is called hidden state in the Recurrent Neural Network



RNN - Limitation

1. The Vanishing Gradient Problem

RNNs process data sequentially, meaning they pass information from one step to the next in a "chain." During training, the model uses backpropagation to update its weights.

2. Sequential Bottleneck (Lack of Parallelization)

RNNs must wait for the computation of the previous hidden state (h_{t-1}) before they can compute the current state (h_t)

3. The "Lost in the Middle" Phenomenon

As the sequence length increases, the hidden state (the model's "memory") gets overwritten by new information

Transformers Vs RNN

Feature	RNN Architecture	Transformer Architecture
Processing	Sequential (One by one)	Parallel (All at once)
Memory	Recurrence (Hidden States)	Self-Attention (Direct Links)
Long-Range	Struggles (Vanishing Gradient)	Excellent (Constant path length)
Speed	Slow to train	Very fast to train

Revision : Dot Product

Dot product in matrix main rules :

$$\begin{bmatrix} a & b \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = [ax + by]$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} w & x \\ y & z \end{bmatrix} = \begin{bmatrix} aw + bx & ax + bz \\ cw + dx & cx + dz \end{bmatrix}$$

Revision : Dot Product

Example

$$\begin{matrix} A \\ B \\ C \\ D \\ E \\ F \end{matrix} \begin{bmatrix} 1.5 & 1.2 & 2.3 & \dots & -0.4 & 2.4 \\ 3.4 & 0.7 & -4.3 & \dots & 6.1 & 4 \\ 1.2 & 5.3 & 1.3 & \dots & 9.2 & 1.3 \\ 4.5 & 6.4 & 1.8 & \dots & 1.3 & 1.6 \\ 6.7 & -1.1 & 2.5 & \dots & 4.4 & -2.4 \\ 2.3 & 1.3 & 2.1 & \dots & -1.2 & 8.8 \end{bmatrix}_{6 \times 512} \cdot \begin{bmatrix} 0.5 & 7.2 & 2.3 & 8.4 & -0.4 & 2.4 \\ 3.4 & -0.7 & -5.3 & 3.1 & 1.1 & 2 \\ 1.2 & 0.3 & 8.3 & 4.3 & 0.2 & 1.3 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0.7 & -1.1 & 2.5 & 9.1 & 4.1 & -2.4 \\ 3.3 & 1.3 & 2.1 & 0.2 & -1.2 & 8.8 \end{bmatrix}_{512 \times 6}$$

The result of the dot product of these two matrices is a matrix with the shape of (6×6)

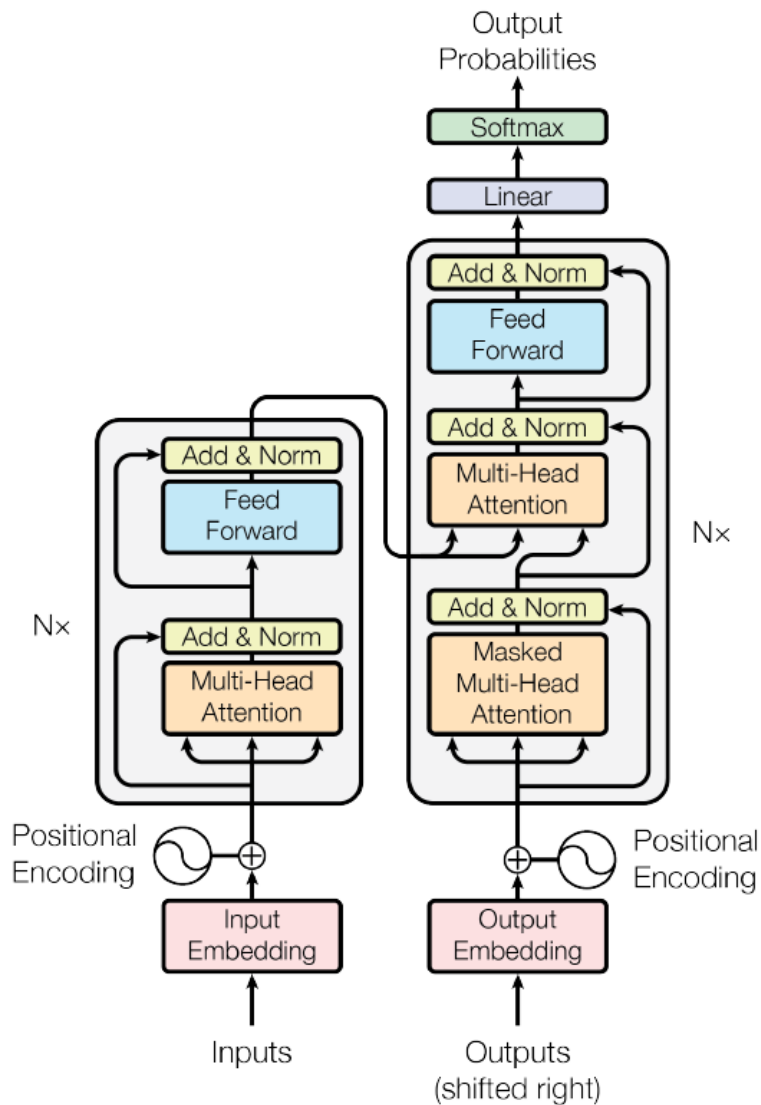


Figure 1: The Transformer - model architecture.

The original architecture consists of two main sections:

- **The Encoder (The Reader):** Its job is to ingest the input sequence (like a sentence in English) and convert it into a rich mathematical representation that captures the meaning and context of every word.
- **The Decoder (The Writer):** It takes the representation from the encoder and uses it to generate an output sequence (like the translated sentence in French), one token at a time.

Why It Changed Everything

The Transformer architecture solved the two biggest problems in previous AI models:

- 1. Parallelization:** Because it doesn't process word-by-word, it can be trained on massive GPUs in parallel, making it possible to train on the entire internet's worth of data.
- 2. Long-Range Dependencies:** Traditional models would "forget" the beginning of a long sentence by the time they reached the end. Transformers maintain a "perfect memory" of all parts of the sequence, no matter how long it is.

Input Embedding

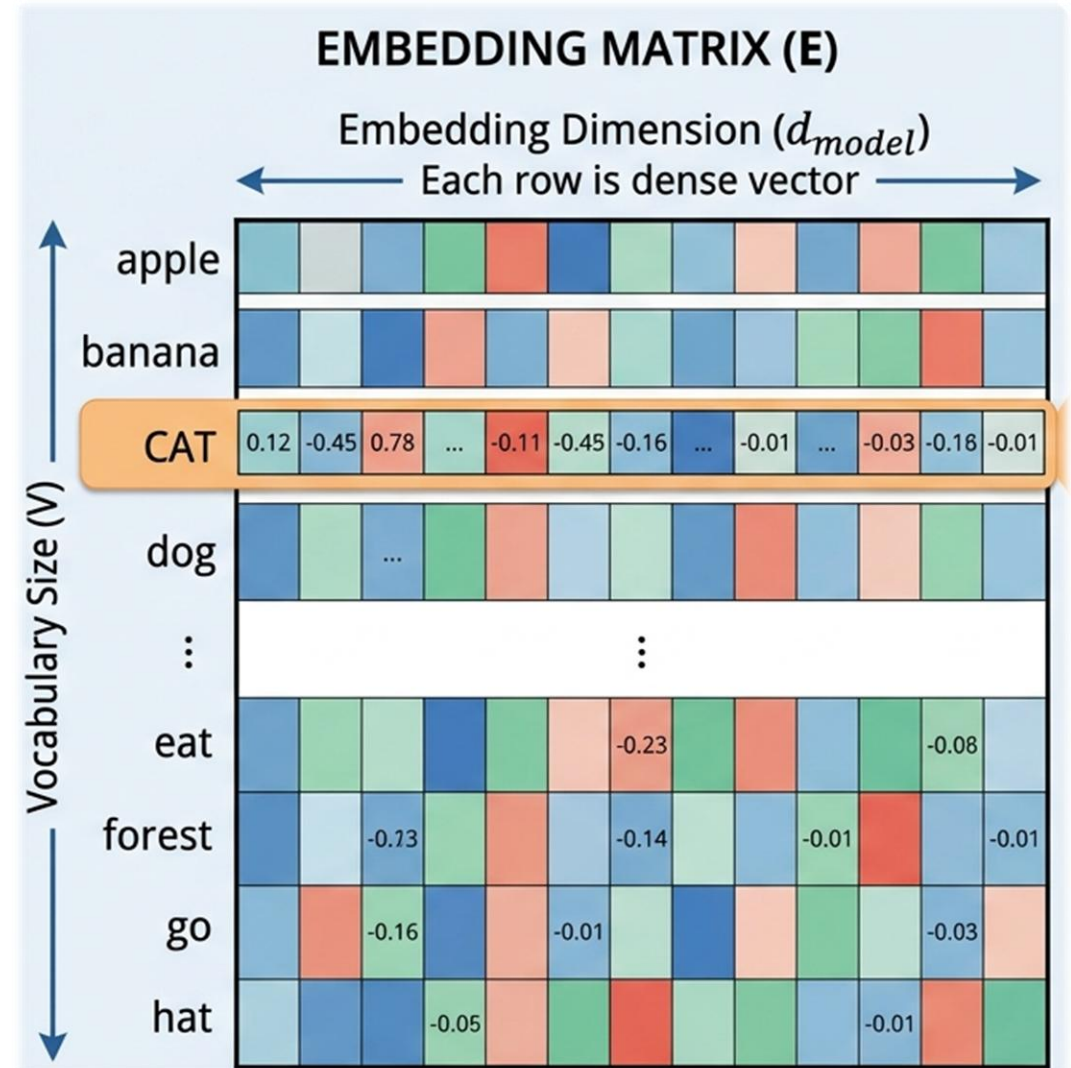
The input sequence is first converted into dense representations using a learned **Embedding Matrix** of shape $(V \times d_{model})$. This stage transforms discrete token IDs into continuous vectors, allowing the model to project semantic features into a high-dimensional latent space for subsequent processing.

Embedding Matrix

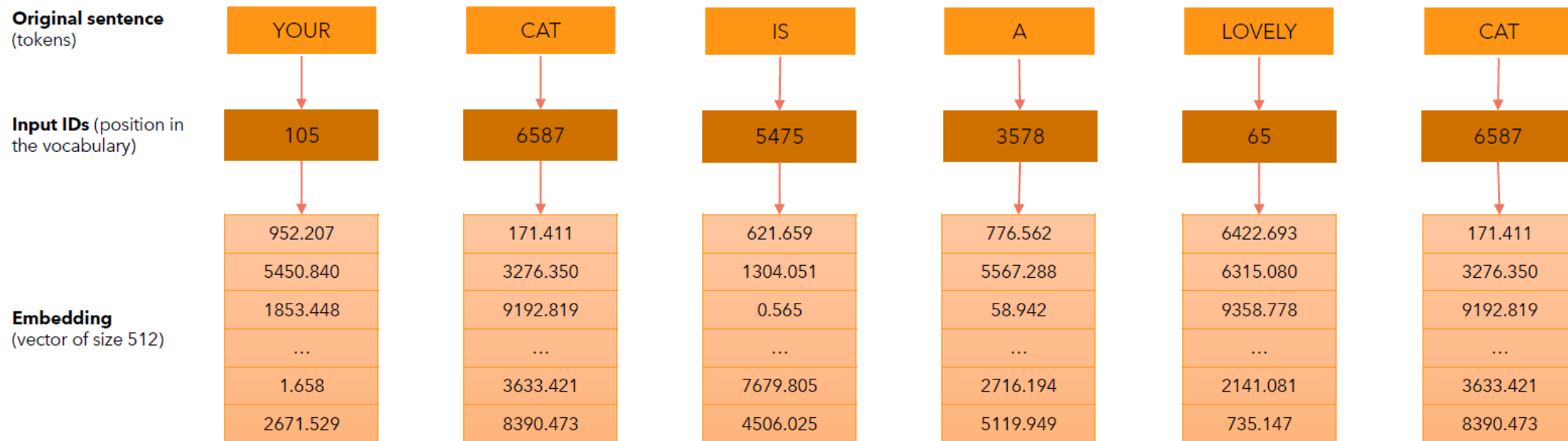
Embedding Matrix (Storage): This is the core matrix has the shape of $(V \times d_{model})$.

Each row corresponds to one word index.

in standard Transformer architecture $d_{model} = 512$.



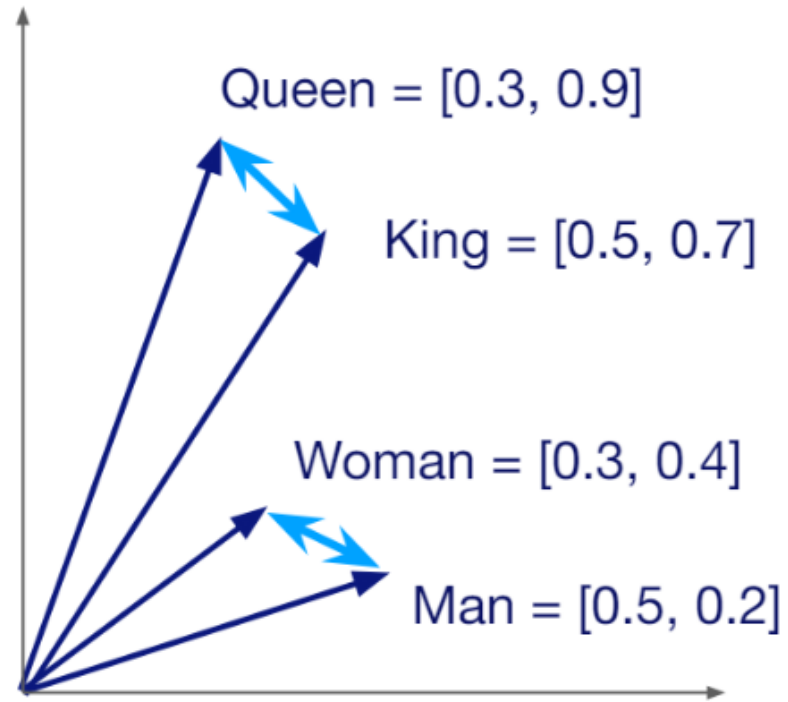
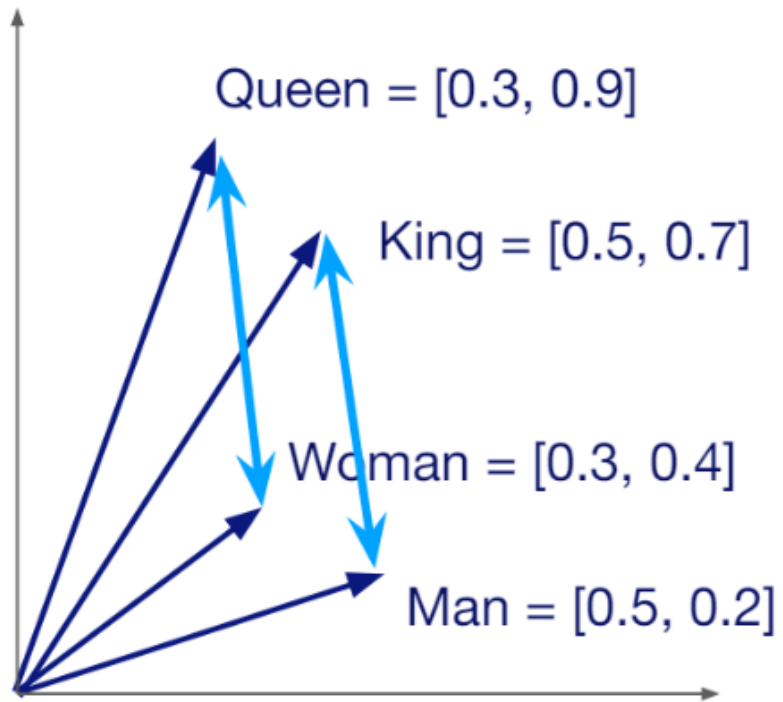
Tokenization



We define $d_{\text{model}} = 512$, which represents the size of the embedding vector of each word

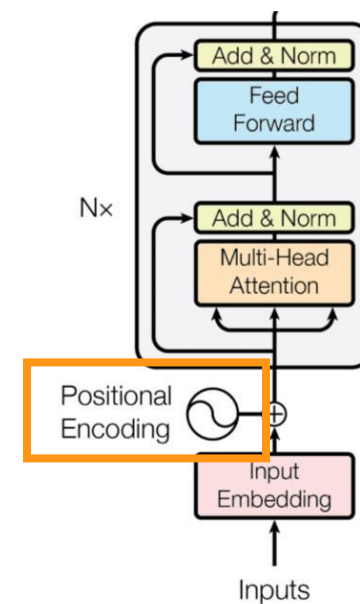
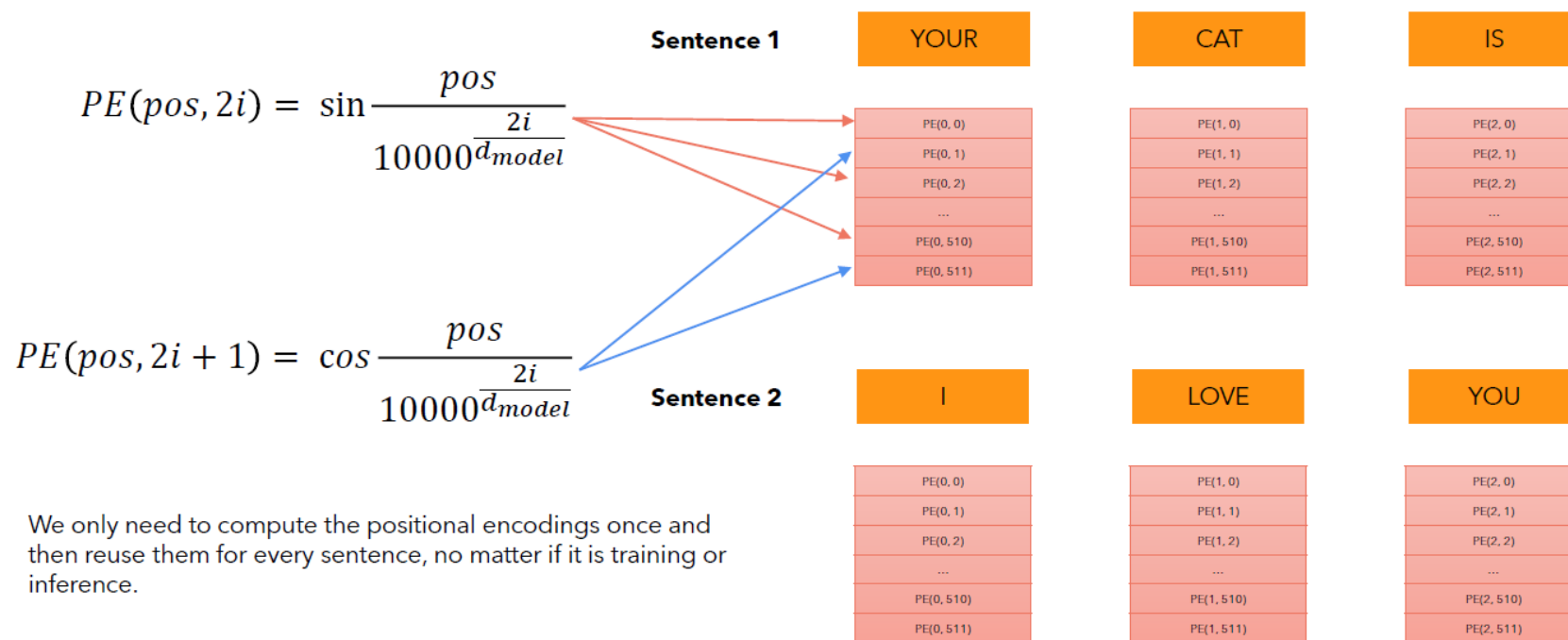
Embedding vectors

Example about embedding vectors with two dimensions



Positional Encoding

- We want each word to carry some information about its position in the sentence.
- We want the model to treat words that appear close to each other as "close" and words that are distant as "distant".



Tokenization + Embedding + positional encoding

Original sentence	YOUR	CAT	IS	A	LOVELY	CAT
Embedding (vector of size 512)	952.207	171.411	621.659	776.562	6422.693	171.411
	5450.840	3276.350	1304.051	5567.288	6315.080	3276.350
	1853.448	9192.819	0.565	58.942	9358.778	9192.819
	...	...	...	...	...	...
	1.658	3633.421	7679.805	2716.194	2141.081	3633.421
	2671.529	8390.473	4506.025	5119.949	735.147	8390.473
Position Embedding (vector of size 512). Only computed once and reused for every sentence during training and inference.	+	+	+	+	+	+
	...	1664.068	...	...	...	1281.458
	...	8080.133	...	...	...	7902.890
	...	2620.399	...	...	...	912.970
	...	...	...	...	...	3821.102
	...	9386.405	...	...	...	1659.217
Encoder Input (vector of size 512)	...	3120.159	...	...	...	7018.620
	=	=	=	=	=	=
	...	1835.479	...	...	...	1452.869
	...	11356.483	...	...	...	11179.24
	...	11813.218	...	...	...	10105.789
	...	...	...	...	...	...
	...	13019.826	...	...	...	5292.638
	...	11510.632	...	...	...	15409.093
	...	...	...	...	...	...

Stage one end

At this stage, the input sequence has been transformed into a feature matrix $X \in \mathbb{R}^{n \times d_{model}}$ where n is the sequence length and d_{model} is the embedding dimension

$$X = \textit{Embedding} + \textit{Positional Encoding}$$

Each row vector x_i represents a token that has been enriched with both semantic meaning (via the embedding matrix) and positional awareness (via positional encodings).

Now this matrix is the input for the Transformer model

Single head – self attention

Self-Attention is the core mechanism that allows a Transformer to understand the context of a word by looking at the other words in the same sentence. Unlike older models that processed words one by one in a fixed order, Self-Attention allows the model to process all words simultaneously and determine which ones are most relevant to each other.

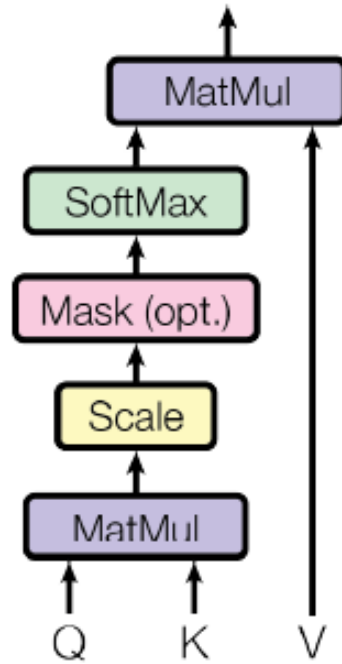
For example:

1. **Apple** released new iPhone ! → (released, iPhone → **Apple** is company)
2. This **Apple** is red and delicious → (red, delicious → **Apple** is fruit)

Single head – self attention $h = 1$

mathematical notation

Scaled Dot-Product Attention



And:

$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V$$

$$X \in \mathbb{R}^{n \times d_{model}}$$

$$W^Q, W^K, W^V \in \mathbb{R}^{d_{model} \times d_k}$$

$$Q, K, V \in \mathbb{R}^{n \times d_k}$$

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$d_k = \frac{d_{model}}{h}$$

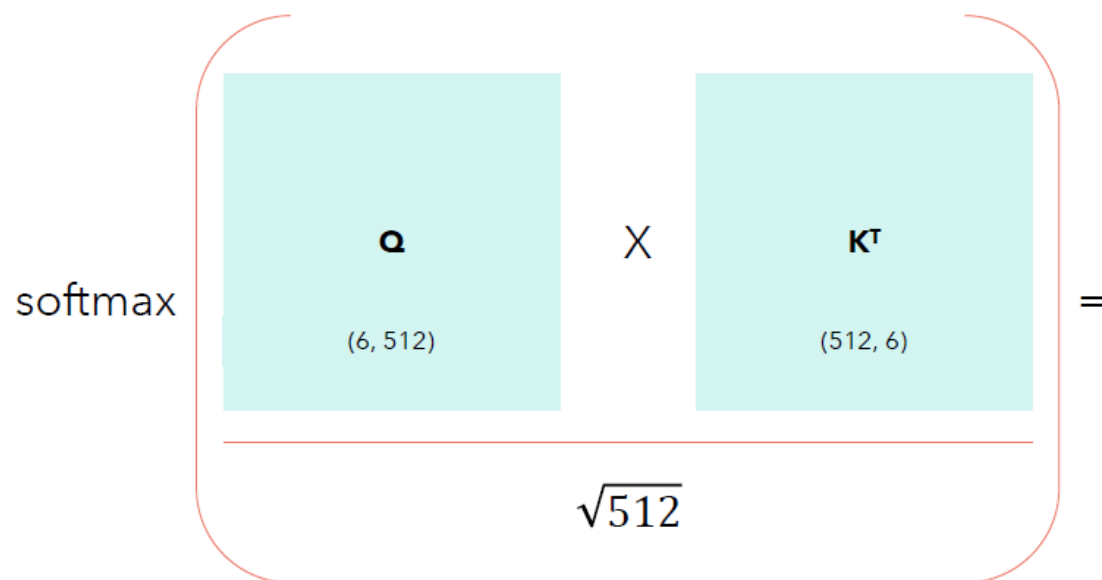
Single head – self attention $h = 1$

Self-Attention allows the model to relate words to each other.

In this simple case we consider the sequence length **seq** = 6 and $\mathbf{d}_{\text{model}} = \mathbf{d}_k = 512$.

The matrices **Q**, **K** and **V** are just the input sentence.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



	YOUR	CAT	IS	A	LOVELY	CAT	Σ
YOUR	0.268	0.119	0.134	0.148	0.179	0.152	1
CAT	0.124	0.278	0.201	0.128	0.154	0.115	1
IS	0.147	0.132	0.262	0.097	0.218	0.145	1
A	0.210	0.128	0.206	0.212	0.119	0.125	1
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174	1
CAT	0.195	0.114	0.203	0.103	0.157	0.229	1

* all values are random.

* for simplicity I considered only one head, which makes $\mathbf{d}_{\text{model}} = \mathbf{d}_k$.

(6, 6)

Single head – self attention $h = 1$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	0.119	0.134	0.148	0.179	0.152
CAT	0.124	0.278	0.201	0.128	0.154	0.115
IS	0.147	0.132	0.262	0.097	0.218	0.145
A	0.210	0.128	0.206	0.212	0.119	0.125
LOVELY	0.146	0.158	0.152	0.143	0.227	0.174
CAT	0.195	0.114	0.203	0.103	0.157	0.229

(6, 6)

$\times$

V

(6, 512)

$=$

Attention

(6, 512)

Each row in this matrix captures not only the meaning (given by the embedding) or the position in the sentence (represented by the positional encodings) but also each word's interaction with other words.

Problems with Single head – self attention

In a sentence, a word usually has multiple roles.

For example, in the sentence "The cat sat on the mat because **it** was soft," the word "it" needs to:

- **Semantic Link:** Realize "it" refers to the "mat" (not the cat).
- **Grammatical Link:** Realize "it" is the subject of the verb "was."

A single head might focus entirely on the grammar and "miss" the semantic connection to the mat. It essentially forces the model to choose one "view" of the data, which leads to a loss of information.

The Solution (Multi-Head Attention):

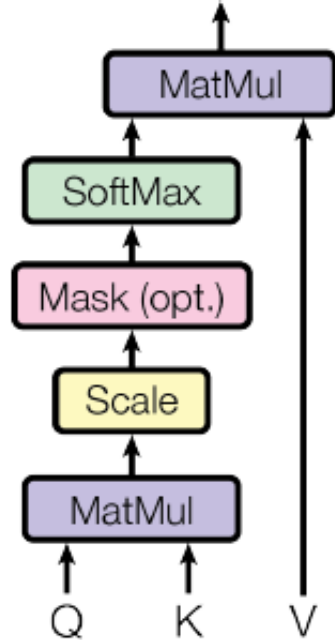
By using multiple heads, the model can look at the sentence in several ways at the same time:

- **Head 1** focuses on grammar.
- **Head 2** focuses on vocabulary/meaning.
- **Head 3** focuses on the relationship between distant words.

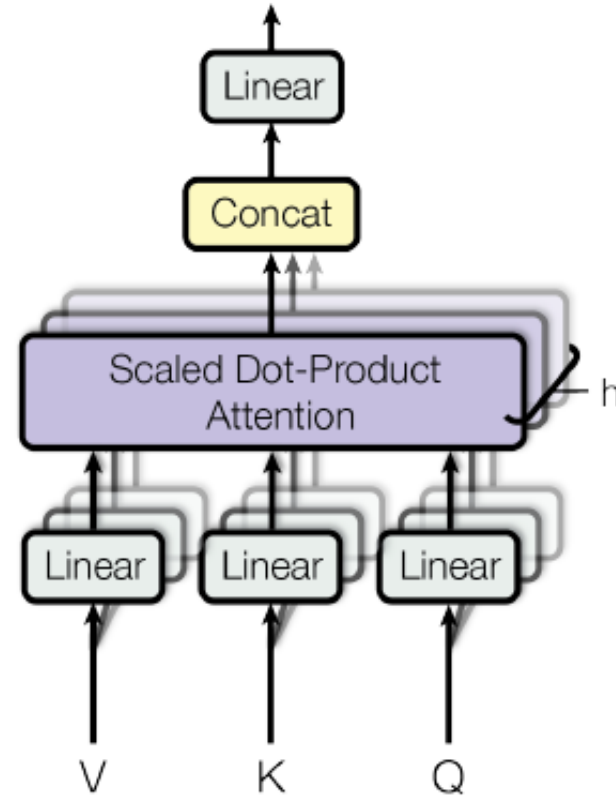
By combining these different "points of view," the model gets a much deeper and more accurate understanding of the text.

Multi head – self attention $h = N$ mathematical notation

Scaled Dot-Product Attention



Multi-Head Attention



Multi head – self attention $h = N$ mathematical notation

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$MultiHead(Q, K, V) = Concat(head_1 \dots head_n)W^O$$

$$head_i = Attention(Q, K, V)$$

$$Q = X W^Q$$

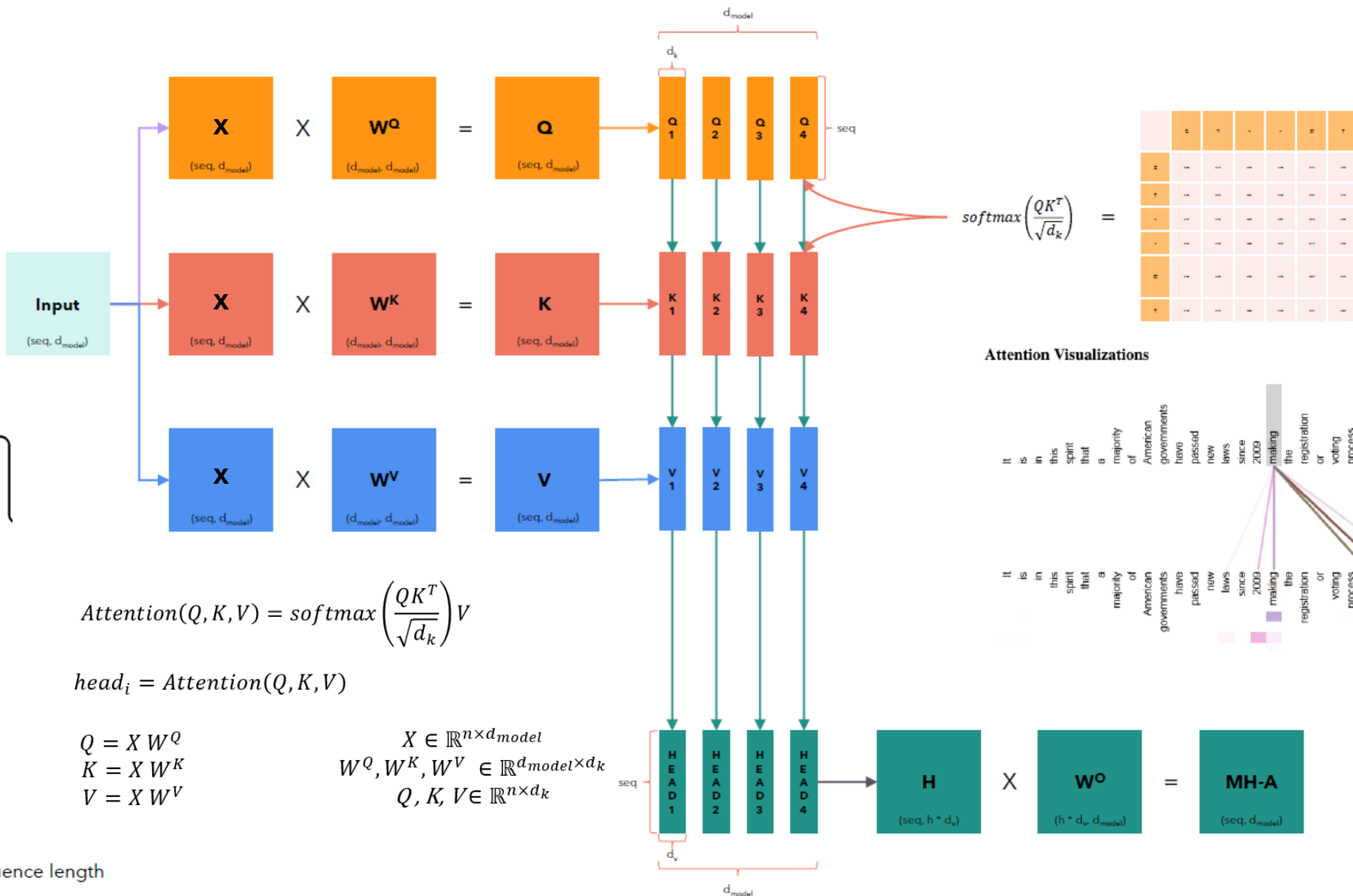
$$K = X W^K$$

$$V = X W^V$$

$$X \in \mathbb{R}^{n \times d_{model}}$$

$$W^Q, W^K, W^V \in \mathbb{R}^{d_{model} \times d_k}$$

$$Q, K, V \in \mathbb{R}^{n \times d_k}$$



seq = sequence length

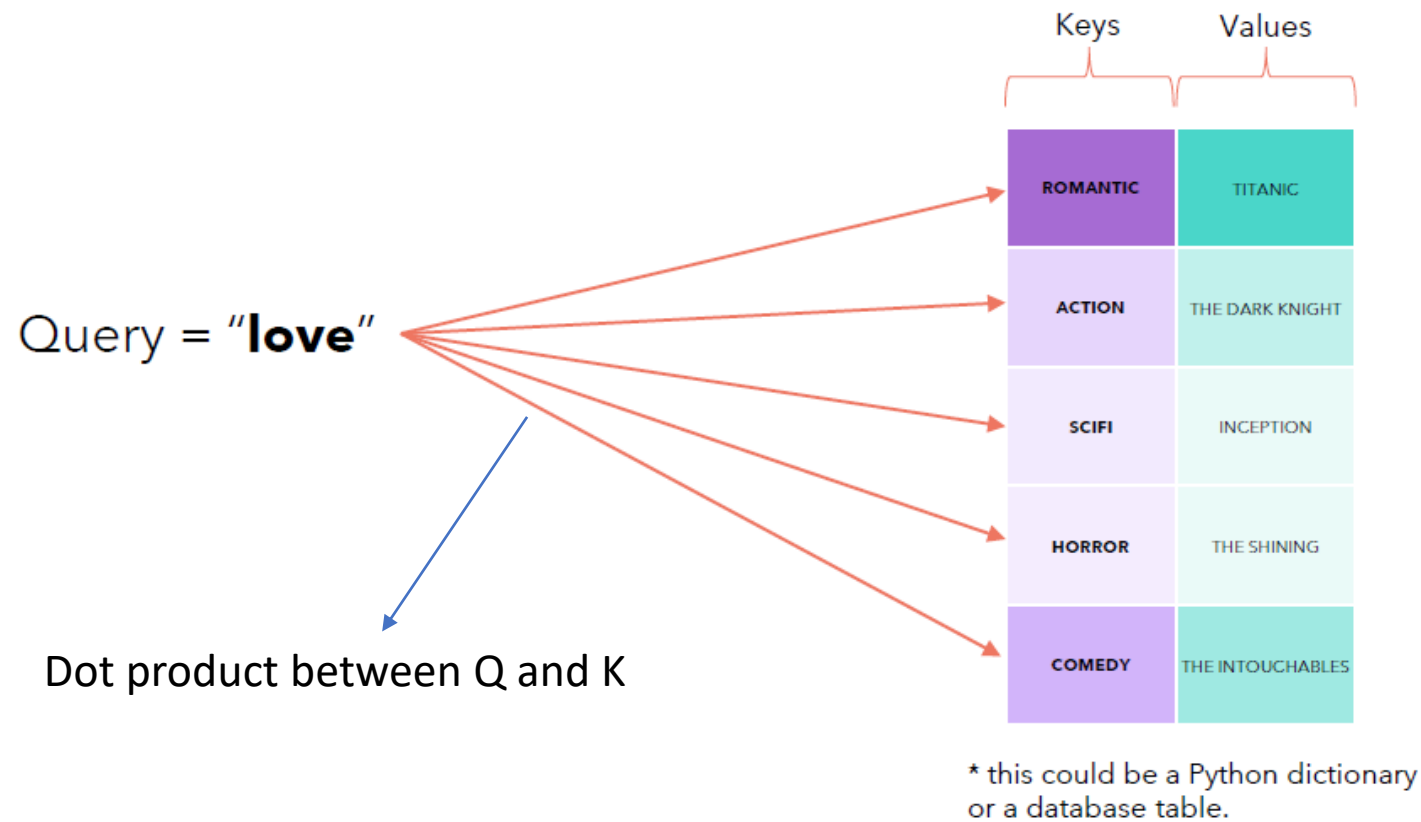
d_{model} = size of the embedding vector

h = number of heads

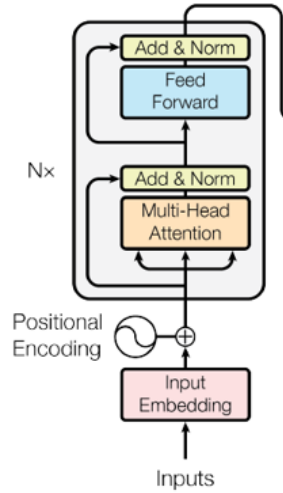
$d_k = d_v$ = d_{model} / h

Why query, keys and values?

The Internet says that these terms come from the database terminology or the Python-like dictionaries.



Add & Norm



$$A = MHA(X)$$

A : Output of $MHA(X)$

X : Original input matrix

Add :

$$Z = A + X$$

Because we need to keep the information from the original word and not only depend on the attention

norm :

$$Y = LayerNorm(Z)$$

Make the training more stable

$$Add \ \& \ Norm(X, A) = LayerNorm(A + X)$$

Feed Forward NN

$$\text{output} = F(Y)$$

Where F is a Dense Neural network applied for each word vector separately and for all word in parallel to make the word vector more stable and more accurate

Then make another *Add & Norm* then we have the output of the Encoder

All the encoder process is repeated multiple times .

Decoder Architecture

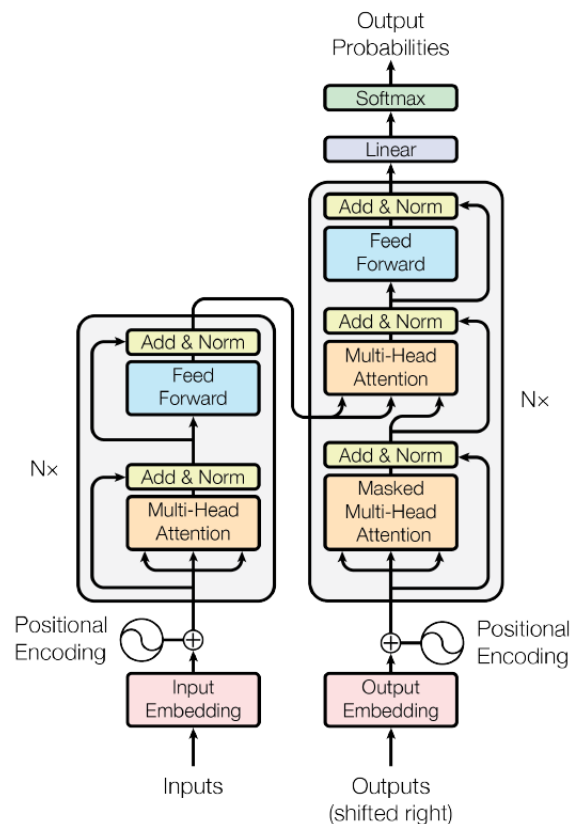


Figure 1: The Transformer - model architecture.

All these structures:

1. Embedding
2. positional encoding
3. Add & Norm
4. Feed Forward

Are the same concept we see in the **Encoder**

The new block we will see the **Masked Multi-head Attention**
And the other multi-head Attention is have the same idea from the encoder but it take the Q from the Masked Multi head Attention and K,V from the output of the encoder

Decoder

To understand the **Decoder**, the best way is to view it as the "Generative" half of the Transformer. While the Encoder builds a map of the input, the Decoder uses that map to predict the next word in a sequence, one by one.

Think of the Encoder as a **translator listening** to a sentence, and the Decoder as the **translator speaking** the result.

1. The Decoder's Two Main Goals

Unlike the Encoder, which looks at everything at once, the Decoder has two specific constraints:

- 1.Auto-regressive Generation:** It produces one word at a time. The word it generates now becomes part of the input for the next word.
- 2.Contextual Awareness:** It must constantly look back at the Encoder's output to make sure it stays "on topic."

Masked self- Attention (The “no peeking Rule”)

In the Encoder, words can look at the future (words to their right). In the Decoder, we can't let the model "see" the answer while it's training.

It ensures the model learns to predict based only on what has already been said.

What is Masked Multi-head Attention?

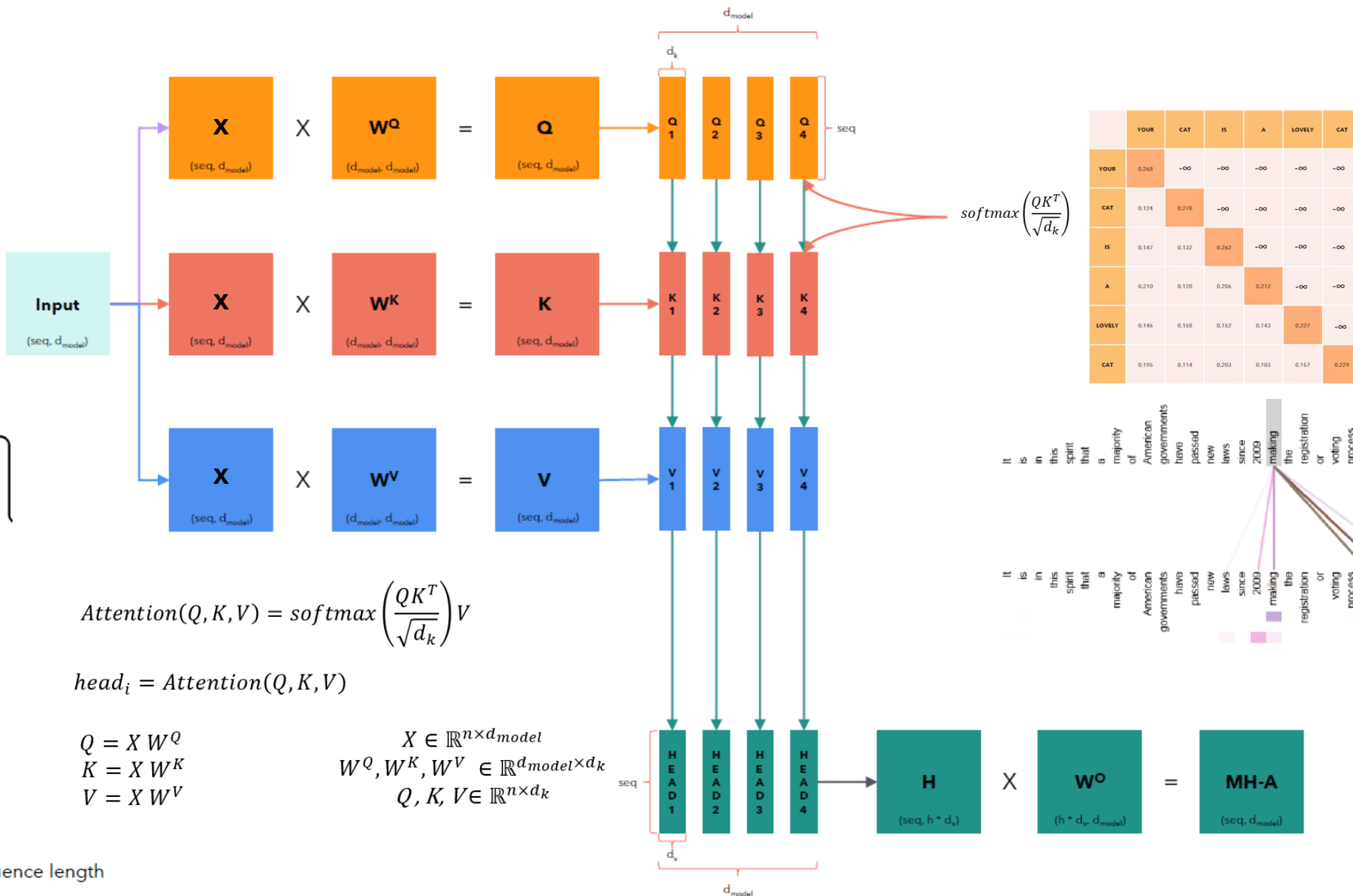
$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

	YOUR	CAT	IS	A	LOVELY	CAT
YOUR	0.268	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
CAT	0.124	0.278	$-\infty$	$-\infty$	$-\infty$	$-\infty$
IS	0.147	0.132	0.262	$-\infty$	$-\infty$	$-\infty$
A	0.210	0.128	0.206	0.212	$-\infty$	$-\infty$
LOVELY	0.146	0.158	0.152	0.143	0.227	$-\infty$
CAT	0.195	0.114	0.203	0.103	0.157	0.229

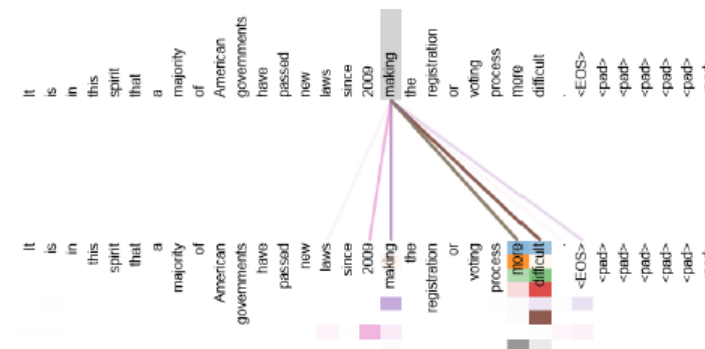
Our goal is to make the model causal: it means the output at a certain position can only depend on the words on the previous positions. The model **must not** be able to see future words.

All the values above the diagonal are replaced with $-\infty$ before applying the softmax, which will replace them with **zero**.

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$



	YOUR	CAT	IS	A	LOVELY	CAT
YOUR						
CAT	0.268					
IS	0.124	0.278				
A	0.147	0.132	0.262			
LOVELY	0.210	0.128	0.206	0.212		
CAT	0.146	0.158	0.152	0.143	0.227	



- seq = sequence length
- d_{model} = size of the embedding vector
- h = number of heads
- $d_k = d_v$ = d_{model} / h

$$MultiHead(Q, K, V) = Concat(head_1 \dots head_h)W^O$$

Revision :

Phase 1: The Encoder (Processing the Input)

1. **Tokenization & Embedding:** The input sentence (e.g., "Your cat is lovely") is broken into tokens. These are converted into vectors using the Embedding Matrix.
2. **Positional Encoding:** Since Transformers don't process words in order, a "position signal" is added to each vector so the model knows where each word sits in the sentence.
3. **Self-Attention Layers:** The words "talk" to each other. The model identifies that "lovely" refers to "cat."
4. **"Final Output:** The Encoder produces a stack of vectors that represent the Context of the entire sentence. This information is sent over to the **Decoder**.

Revision :

Phase 2: The Decoder (Generating the Output)

5. **The Start Signal:** The Decoder is given a "Start of Sentence" <SOS> token to begin.
6. **Masked Self-Attention:** The Decoder looks at the words it has already generated. It uses a Mask to make sure it doesn't "cheat" by looking at future words during training.
7. **Encoder-Decoder Attention (The Bridge):** This is the crucial step. The Decoder "queries" the Encoder's output. It looks at the context of the original English sentence to decide which word to translate next.
8. **Feed-Forward & Linear Layer:** The data is processed through a neural network and projected onto the size of the Vocabulary (e.g., 50,000 possible words).
9. **Softmax (The Decision):** The model calculates probabilities and picks the most likely next word.

Revision :

Phase 3: The Auto-regressive Loop

10. Feedback: The word chosen in Step 9 is fed back into the Decoder at the bottom.

11. Repeat: The process repeats (Steps 6–10) until the model generates an <EOS> (End of Sentence) token.

Inferencing

Example :



1. **Context Extraction:** The Encoder transforms the input into high-dimensional vectors.
2. **Initialization:** The Decoder starts with a $< SOS >$ (Start of Sentence) token.
3. **Cross-Attention:** The Decoder "queries" the Encoder's memory to find relevant information.
4. **Token Selection:** A Linear layer and softmax function choose the most likely next word.
5. **Iteration:** The new word is appended to the Decoder's input, and the cycle restarts.

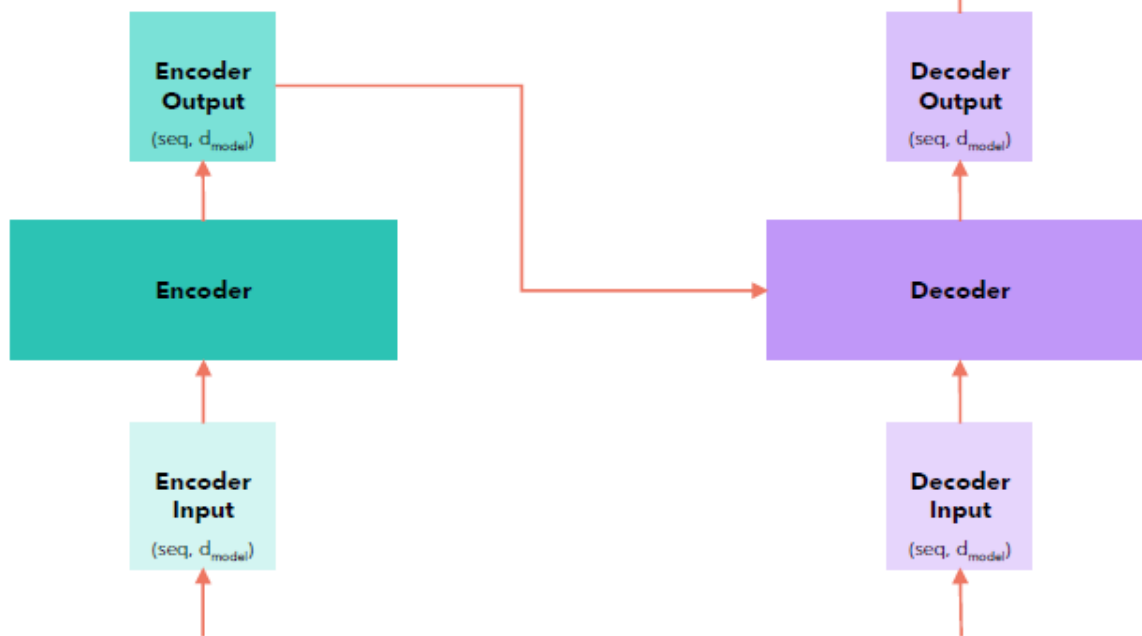
We selected, at every step, the word with the maximum softmax value. This strategy is called **greedy** and usually does not perform very well.

A better strategy is to select at each step the top B words and evaluate all the possible next words for each of them and at each step, keeping the top B most probable sequences. This is the **Beam Search** strategy and generally performs better.

Inference

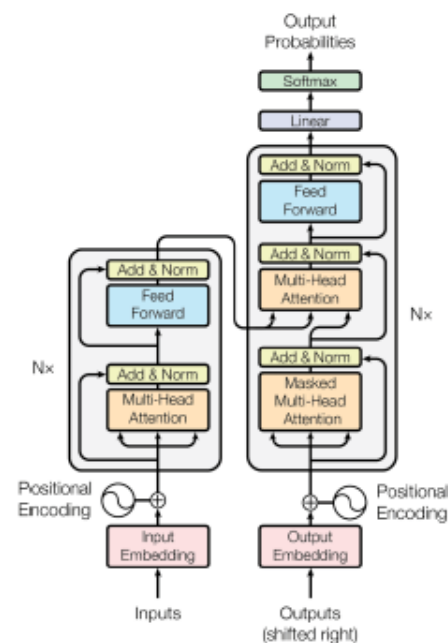
Time Step = 1

<SOS>I love you very much<EOS>



We select a token from the vocabulary corresponding to the position of the token with the maximum value.

The output of the last layer is commonly known as **logits**



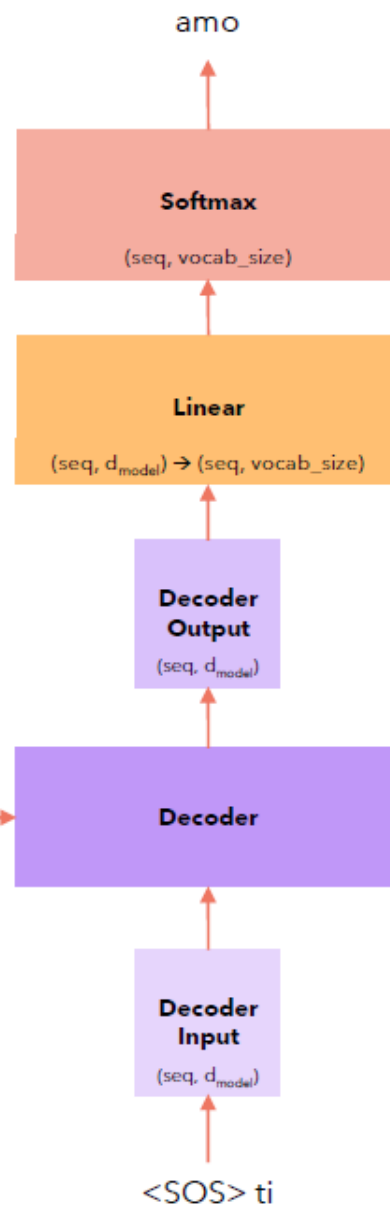
* Both sequences will have same length thanks to padding

Inference

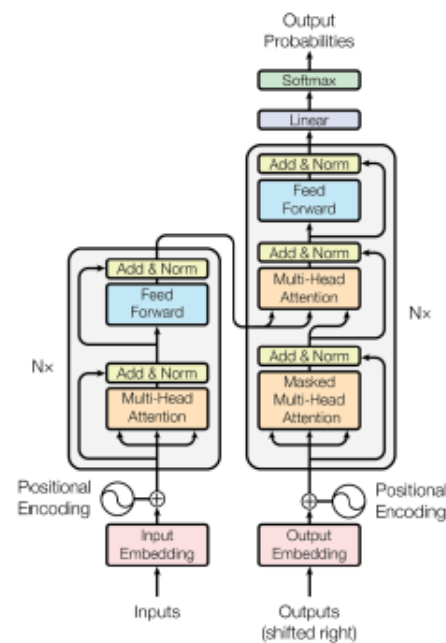
Time Step = 2

<SOS>I love you very much<EOS>

Use the encoder output from the first time step



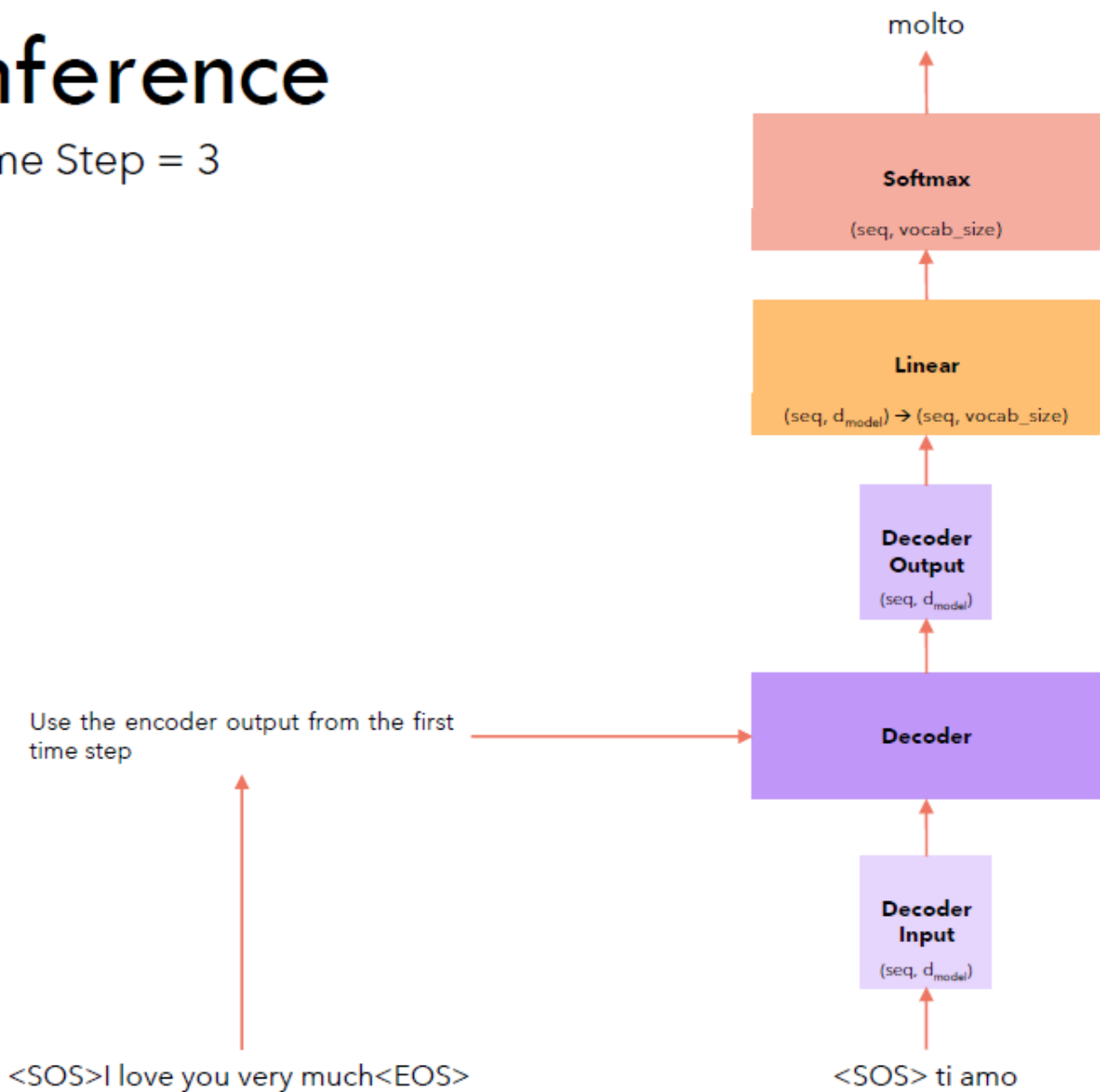
Since decoder input now contains **two** tokens, we select the softmax corresponding to the second token.



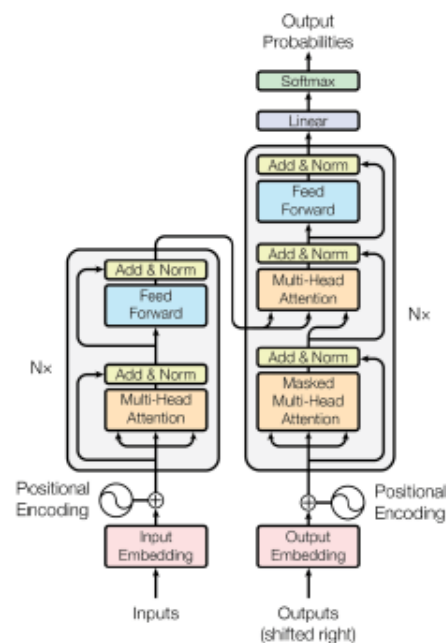
Append the previously output word to the decoder input

Inference

Time Step = 3



Since decoder input now contains **three** tokens, we select the softmax corresponding to the third token.



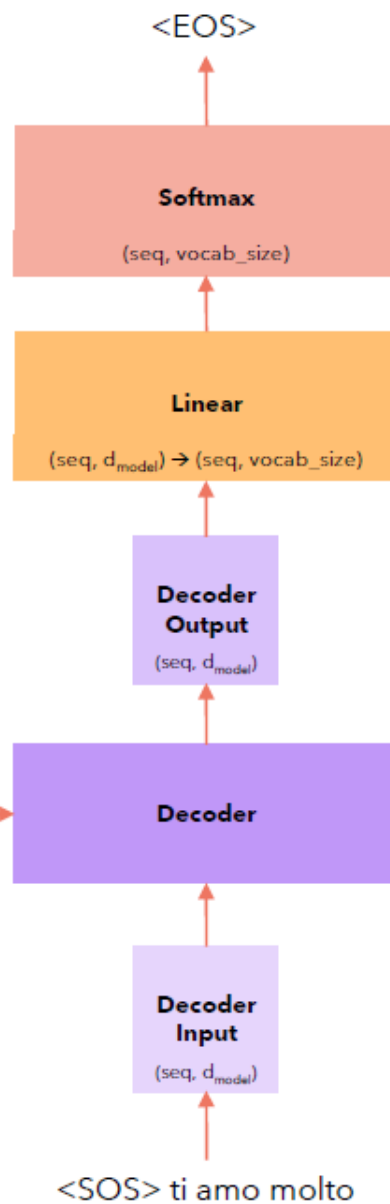
Append the previously output word to the decoder input

Inference

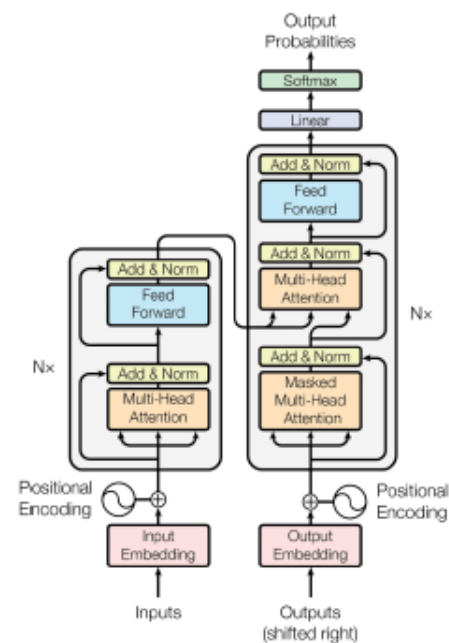
Time Step = 4

<SOS>I love you very much<EOS>

Use the encoder output from the first time step



Since decoder input now contains **four** tokens, we select the softmax corresponding to the fourth token.



Append the previously output word to the decoder input